

Evaluación Final Transversal

Ejecución práctica

Contexto

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
FPY1101	Fundamentos de programación	4 horas	40%

1.Situación Evaluativa 1:

X	Ejecución práctica
---	--------------------

2.Instrucciones Evaluación Final Transversal

La biblioteca digital **ReadCloud** requiere un programa en Python para administrar su catálogo de libros y la disponibilidad de copias para préstamo. Todo el comportamiento del sistema debe organizarse en funciones bien definidas. El programa incluye un menú interactivo, validaciones de entrada y una separación clara entre la lógica de cada función y las decisiones del programa principal.

1. Datos que debe manejar el sistema

El sistema trabaja con **dos diccionarios relacionados**, ambos identificados por el mismo código de libro como clave. Estos diccionarios deben existir desde que el programa inicia y permanecer disponibles durante toda la ejecución.

Estos diccionarios deben crearse en el programa principal y pasarse como argumento a todas las funciones que necesiten leerlos o modificarlos (aunque estas no los soliciten en el enunciado). No está permitido acceder a ellos como variables globales dentro de las funciones. También es importante indicar que cada estructura de repetición (while, for) debe utilizarse según su propósito natural (dependiendo de si la condición de parada cambia durante la ejecución o si el volumen de iteraciones se conoce de antemano). Usar una estructura fuera de ese contexto será considerado un error de diseño y no cumplirá con el estándar de la evaluación.

Diccionario libros

Contiene la información descriptiva de cada libro. La **clave** es el código de libro y el **valor** es una lista con los siguientes campos, en este orden:

Campo	Qué representa	Restricciones de validación
"titulo"	Título del libro	No debe contener solo espacios en blanco ni estar vacío
"autor"	Nombre del autor	No debe contener solo espacios en blanco ni estar vacío
"genero"	Género literario del libro	No debe contener solo espacios en blanco ni estar vacío
"año"	Año de publicación	Número entero mayor que cero
"editorial"	Editorial que publicó el libro	No debe contener solo espacios en blanco ni estar vacío
"es_novedad"	Indica si el libro es una novedad editorial	El usuario ingresa 's' o 'n'. El sistema lo convierte a True o False

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
libros = {
    'L001': ['Sombras del Sur', 'A. Rojas', 'novela', 2019,
    'AndesPress', False],
    'L002': ['Python en Ruta', 'M. Diaz', 'tecnología', 2023,
    'CodeBooks', True],
    'L003': ['Mar y Viento', 'C. Silva', 'poesía', 2017, 'Litoral',
    False],
    'L004': ['Historia Breve', 'J. Pérez', 'historia', 2015,
    'Cronos', False],
    'L005': ['Mundos Lejanos', 'L. Torres', 'ciencia ficción', 2021,
    'Orión', True],
    'L006': ['Cocina Simple', 'R. Soto', 'cocina', 2018, 'Sabores',
    False],
    ...
}
```

Diccionario prestamos

Contiene la información operativa de cada libro. La **clave** es el mismo código de libro y el **valor** es una lista con los siguientes dos campos:

Campo	Qué representa	Restricciones de validación
"precio_multa"	Valor de la multa por día de atraso en pesos	Número entero mayor que cero

"copias_disponibles"	Cantidad de copias disponibles para préstamo	Número entero mayor o igual a cero
----------------------	--	------------------------------------

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
prestamos = {
    'L001': [500, 4],
    'L002': [700, 0],
    'L003': [300, 10],
    'L004': [400, 2],
    'L005': [600, 1],
    'L006': [350, 6],
    ...
}
```

2. Lo que debe hacer el sistema

El sistema se controla desde un menú que aparece en pantalla cada vez que el usuario termina una acción. El usuario elige una opción numérica, el programa ejecuta la tarea correspondiente y vuelve a mostrar el menú. Esto se repite hasta que el usuario elige salir. Si el usuario ingresa un valor que no corresponda a ninguna opción válida, el sistema muestra el mensaje "Debe seleccionar una opción válida" y vuelve a mostrar el menú.

Para la lectura de la opción del menú:

Define una función llamada **leer_opcion()**. No recibe parámetros. Solicita al usuario que ingrese una opción, valida que el valor ingresado sea un número entero y que esté dentro del rango de opciones válidas del menú, y retorna ese valor entero. Si el usuario ingresa un dato que no es un entero debe manejarlo mediante excepciones.

```
===== MENÚ PRINCIPAL =====
1. Copias por género
2. Búsqueda de libros por rango de multa
3. Actualizar multa de libro
4. Agregar libro
5. Eliminar libro
6. Salir
=====
```

Opción 1 — Copias por género

El sistema solicita al usuario el nombre de un género literario (por ejemplo: novela, tecnología o poesía). La búsqueda **no distingue entre mayúsculas y minúsculas**, por lo que "novela" y "NOVELA" deben producir el mismo resultado. El sistema recorre el diccionario **libros** identificando todos los libros que pertenezcan a ese género. Por cada libro encontrado, se debe buscar su código en el diccionario **prestamos**, extraer la cantidad de copias disponibles (el segundo elemento de la lista) y acumularla en un total. Una vez procesados todos los libros, se debe mostrar dicho total acumulado en pantalla.

Para implementar esta opción:

Define una función llamada **`copias_genero(genero)`**. Recibe el género como parámetro, **no retorna ningún valor** y muestra el resultado directamente por pantalla.

Opción 2 — Búsqueda de libros por rango de multa

El sistema solicita al usuario un valor mínimo y un valor máximo de multa. Luego recorre el diccionario **`prestamos`** y construye una lista con todos los libros que: (a) tengan una multa dentro del rango ingresado, y (b) tengan copias disponibles (copias distinto de cero). Cada elemento de la lista tiene el formato "Título--Código". Los resultados se muestran ordenados **alfabéticamente por título**. Si no hay libros que cumplan las condiciones, el sistema muestra: *"No hay libros en ese rango de multa."*

Restricciones de entrada:

El valor mínimo y máximo de multa deben ingresarse como valores enteros. Esta validación ocurre en el **programa principal**, antes de llamar a la función. Como el usuario puede ingresar cualquier tipo de dato, debe utilizarse manejo de excepciones. Si el dato ingresado no es un entero válido, el sistema muestra "Debe ingresar valores enteros" y vuelve a solicitar ambos valores.

Para implementar esta opción:

Define una función llamada **`busqueda_multa(multa_min, multa_max)`**. Recibe el valor mínimo y máximo de multa como parámetros (*estos valores deben ser mayores o iguales a cero y el `multa_min` menor o igual al `multa_max`*), **no retorna ningún valor** y muestra los resultados directamente por pantalla.

Opción 3 — Actualizar multa de libro

El sistema solicita al usuario el código del libro y el nuevo valor de multa que se desea asignar. Si el código existe en el diccionario **`prestamos`**, el sistema actualiza su multa. Si el código no existe, informa al usuario. Al terminar, pregunta: *"¿Desea actualizar otra multa (s/n)?"*: si la respuesta es "s", el proceso se repite; si es "n", el programa vuelve al menú principal.

Para implementar esta opción:

Define una función **`buscar_codigo(codigo)`** que recorra el diccionario y retorne True si el código existe, o False si no existe.

Define una función **`actualizar_multa(codigo, nueva_multa)`** que debe:

- Verificar la existencia del código (se recomienda invocar a `buscar_codigo` internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, actualizar el precio en el diccionario y retornar True.
- Si el código no existe, retornar False.

El **programa principal** debe llamar a `actualizar_multa` y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Multa actualizada" si fue True, o "El código no existe" si fue False.

Recordar que la `nueva_multa` debe ser un valor entero positivo y la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 4 — Agregar libro

El sistema solicita al usuario todos los datos del nuevo libro: código, título, autor, género, año, editorial, si es novedad, multa y copias disponibles. Antes de crear el registro, **cada dato es validado de forma independiente**. Si algún dato no cumple su condición, el sistema informa al usuario y **no registra el libro**. Solo cuando todos los datos son válidos y el código no existe previamente, el sistema agrega el registro en ambos diccionarios.

La siguiente tabla resume las condiciones que debe cumplir cada campo:

Campo solicitado	Condición de validación
código	No vacío ni solo espacios en blanco, y que no exista ya en los diccionarios
título	No vacío ni solo espacios en blanco
autor	No vacío ni solo espacios en blanco
género	No vacío ni solo espacios en blanco
año	Número entero mayor que cero
editorial	No vacío ni solo espacios en blanco
es_novedad	El usuario ingresa 's' o 'n'. El sistema almacena True si es 's', False si es 'n'
precio_multa	Número entero mayor que cero
copias_disponibles	Número entero mayor o igual a cero

Para implementar esta opción:

Define una función de validación independiente para cada campo de la tabla anterior. Cada función recibe únicamente el dato a validar, aplica su condición y retorna **True** si es válido o **False** si no lo es. Los mensajes de error **no** se muestran dentro de las funciones de validación.

En el **programa principal**, al elegir esta opción, se solicitan los datos al usuario y se llama a cada función de validación. Si alguna retorna **False**, el programa muestra el mensaje de error correspondiente y no registra el libro.

Solo si todas las validaciones retornan **True**, el programa llama a la función **agregar_libro(codigo, titulo, autor, genero, año, editorial, es_novedad, precio_multa, copias_disponibles)**, que agrega el registro en ambos diccionarios y retorna **True**. Si el código ya existía, retorna **False**. El programa principal muestra: "Libro agregado" o "El código ya existe" según corresponda.

Opción 5 — Eliminar libro

El sistema solicita el código del libro que se desea eliminar. Si el código existe, elimina el registro en **ambos diccionarios** (libros y prestamos) e informa que la operación fue exitosa. Si el código no existe, informa al usuario.

Para implementar esta opción:

Reutiliza la función **buscar_codigo(codigo)** definida anteriormente para verificar la existencia del libro.

Define una función **eliminar_libro(codigo)** que debe:

- Verificar la existencia del código (se recomienda invocar a **buscar_codigo** internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, eliminar el registro en ambos diccionarios y retornar **True**.
- Si el código no existe, retornar **False**.

El **programa principal** debe llamar a **eliminar_libro** y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Libro eliminado" si fue **True**, o "El código no existe" si fue **False**.

Recordar que la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 6 — Salir

El sistema termina la ejecución de forma limpia. El ciclo del menú se detiene y el programa finaliza mostrando el mensaje: "*Programa finalizado.*"

Para implementar esta opción:

Esta opción no requiere función adicional. El programa principal es responsable de detener el ciclo del menú y mostrar el mensaje de cierre.

3. Ejemplo de ejecución

A continuación, se muestra un ejemplo representativo de cómo debe funcionar el programa. Los datos en **negrita** son valores ingresados por el usuario:

```
===== MENÚ PRINCIPAL =====
1. Copias por género
2. Búsqueda de libros por rango de multa
3. Actualizar multa de libro
4. Agregar libro
5. Eliminar libro
6. Salir
=====
Ingrese opción: 1

Ingrese género a consultar: NOVELA

El total de copias disponibles es: 4


Ingrese opción: 2

Ingrese multa mínima: hola

Debe ingresar valores enteros

Ingrese multa mínima: 300

Ingrese multa máxima: 500

Los libros encontrados son: ['Cocina Simple--L006', 'Historia Breve--L004',
'Mar y Viento--L003', 'Sombras del Sur--L001']
```

Ingrese opción: 3

Ingrese código del libro: S010

Ingrese nueva multa: 800

El código no existe

¿Desea actualizar otra multa (s/n)? : n

Ingrese opción: 4

Ingrese código del libro: L010

Ingrese título: Datos en Vivo

Ingrese autor: P. Morales

Ingrese género: tecnología

Ingrese año de publicación: 2022

Ingrese editorial: BytePress

¿Es novedad? (s/n): s

Ingrese precio de multa: 550

Ingrese copias disponibles: 5

Libro agregado

Ingrese opción: 6

Programa finalizado.